

PART 1 OF 3 • THE BASICS

RAG Fundamentals

Retrieval-Augmented Generation — base concepts, architecture, and end-to-end pipeline



A self-study guide • Casual Hinglish style
Topics covered end-to-end with code, diagrams & intuition

1. Introduction: Why RAG?

LLM bahut powerful hai — par usme 3 fundamental problems hain jo production systems ko break kar deti hain:

- **Knowledge cutoff:** Model ka training data ek date pe freeze ho jata hai. Aaj ka news, kal ka product update, ya tumhari company ka internal doc — model ko kuch nahi pata.
- **Hallucinations:** Model confidently galat jawab deta hai. Plausible lagta hai, par facts wrong hote hain. Ye especially dangerous hai legal, medical, ya enterprise use-cases me.
- **No source attribution:** Model jab kuch bolta hai, tum verify nahi kar sakte ki ye information kaha se aayi. Trust kaise build karoge?
- **Context limits:** Even with 200K+ context windows, har query me poori knowledge base daalna na practical hai, na cost-effective.

RAG (Retrieval-Augmented Generation) in problems ka direct solution hai. Idea simple hai: *generation se pehle, query-relevant information retrieve karo, phir LLM ko vo context ke saath jawab generate karne do.*

■ KEY POINT

RAG = Retrieval (external knowledge fetch karo) + Augmented (LLM prompt me daalo) + Generation (answer banao). Model ko 'open-book exam' dene jaisa hai.

1.1 RAG vs Fine-tuning vs Long-Context

Ye debate har interview me aata hai. Ek-ek karke samjho:

Approach	Best for	Cost	Latency	Updates
RAG	Factual recall, latest data, citations (no retraining)	Low (no training)	Medium (retrieval + LLM)	Easy — just update DB
Fine-tuning	Style, tone, domain language, behavior (customizing data prep)	High (costly data prep)	High (no retrieval)	Hard — retrain required
Long-context	Small/medium docs that fit in window	Very high per query (tokens)	High (long input)	N/A

■ TIP

Production me tum often **RAG + lightweight fine-tuning** combine karte ho — RAG for facts, fine-tuning for tone/format. Ye 'best of both worlds' pattern hai.

1.2 When NOT to use RAG

- Knowledge base bahut chhota hai (e.g., 2-3 documents) → just stuff into context.
- Use-case purely generative hai (creative writing, brainstorming).
- Reasoning-heavy task hai jaha facts kam aur logic zyada chahiye.
- Latency-critical real-time app jaha retrieval overhead unacceptable hai.

2. RAG Architecture: The Big Picture

Ek standard RAG system ki do main pipelines hoti hain:

Pipeline A: Indexing (Offline, one-time / periodic)

```
Raw Documents (PDFs, web pages, DBs)
  ↓
[1] Document Loader → parsed text
  ↓
[2] Text Splitter → chunks
  ↓
[3] Embedding Model → dense vectors
  ↓
[4] Vector Store → indexed for fast retrieval
```

Pipeline B: Query (Online, per request)

```
User Query
  ↓
[1] Embed query (same model as indexing)
  ↓
[2] Similarity search in vector store → top-k chunks
  ↓
[3] Construct prompt: [system + chunks + query]
  ↓
[4] LLM generates answer
  ↓
Response (with citations)
```

■ NOTE

Indexing aur Query pipelines mein **same embedding model** use karna critical hai. Different models = different vector spaces = garbage similarity scores.

2.1 Core Components Quick Reference

Component	Role	Popular choices
Document Loader	Raw files → text	LangChain loaders, Unstructured, LlamaIndex
Splitter	Long text → chunks	RecursiveCharacterTextSplitter, semantic
Embedder	Text → vector	OpenAI, BGE, E5, sentence-transformers
Vector DB	Store + ANN search	FAISS, Chroma, Pinecone, Qdrant, Weaviate
Retriever	Query → relevant chunks	Similarity, MMR, hybrid
LLM	Context → answer	GPT-4, Claude, Llama, Mistral
Orchestrator	Glue	LangChain, LlamaIndex, Haystack

3. Document Loading & Parsing

First step — raw documents ko clean text me convert karna. Format-specific challenges hote hain:

3.1 Common Formats

- **PDF:** Sabse problematic. Text layer present hai to easy, scanned PDFs me OCR chahiye. Tables, multi-column layouts, headers/footers — sab tricky.
- **HTML/Web:** Boilerplate (nav, ads, footer) remove karna padta hai. Tools: BeautifulSoup, trafilatura, readability-lxml.
- **DOCX/PPTX:** python-docx, python-pptx libraries. Generally cleaner than PDF.
- **Markdown:** Easiest — already structured.
- **Structured data (CSV, JSON, DB):** Row-level chunks, ya transform to natural language.

3.2 Code Example: Multi-format Loading

```
from langchain_community.document_loaders import (
    PyPDFLoader, WebBaseLoader, UnstructuredMarkdownLoader
)

# PDF
pdf_docs = PyPDFLoader('paper.pdf').load()

# Web page
web_docs = WebBaseLoader('https://example.com/blog').load()

# Markdown
md_docs = UnstructuredMarkdownLoader('notes.md').load()

# Each Document has: page_content (str) + metadata (dict)
print(pdf_docs[0].metadata)
# {'source': 'paper.pdf', 'page': 0}
```

3.3 Pro-tips for PDF Parsing

- **pypdf / pdfplumber** for text-based PDFs (fast, free).
- **PyMuPDF (fitz)** for better layout preservation.
- **Unstructured / LlamaParse / Docling** — premium parsers that handle tables, headers, and layout intelligently. Worth it for complex docs.
- **Marker / Nougat** — ML-based parsers, especially good for scientific papers.
- For scanned PDFs: **Tesseract OCR, PaddleOCR**, or cloud OCR APIs.

■ ■ WARNING

PDF parsing me **tables** sabse painful hain. Naive text extraction tables ko mess kar deti hai. Either: (1) use table-aware parser, (2) extract tables separately as structured data, or (3) convert tables to markdown/HTML before chunking.

3.4 Metadata Matters!

Metadata = filtering ka backbone. Har chunk ke saath save karo:

- source (filename/URL), page number, section title, author, date
- document type, language, access permissions
- custom tags: product, department, topic

Production me tum ye metadata use karke **metadata filters** apply karoge — e.g., 'sirf 2024 ke docs me search karo' ya 'sirf marketing dept ke docs'.

4. Chunking: The Most Underrated Step

Chunking matlab long documents ko smaller pieces me todna jo embedding aur retrieval ke liye suitable ho. Ye step seedha downstream quality affect karta hai — *garbage chunks in, garbage retrieval out*.

4.1 Why Chunk at All?

- **Embedding model limits:** Most models ka max input 512 tokens (BERT-family) ya 8192 tokens (modern). Long doc ek single vector me reduce karna semantically lossy hai.
- **Retrieval precision:** Chhote chunks zyada specific hote hain → better matching.
- **Context window economy:** Top-k retrieve karne par LLM me kam tokens use honge.
- **Cost:** Smaller chunks = fewer wasted tokens.

4.2 Chunking Strategies

(a) Fixed-size chunking

Simplest. N characters/tokens ke chunks banao, optional overlap ke saath.

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=1000,      # 1000 chars per chunk
    chunk_overlap=200,    # 200 chars overlap
    separator='\n'
)
chunks = splitter.split_text(long_text)
```

- ■ Fast, simple, predictable
- ■ Sentences/paragraphs ko mid-way kaat sakta hai

(b) Recursive character splitting

Ye default choice hai most projects me. Hierarchical separators try karta hai: first paragraphs (\n\n), then lines (\n), then sentences, then words. Coherent units preserve hote hain.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
    separators=['\n\n', '\n', '.', ',', ' ', '']
)
```

(c) Document-structure-aware

Document ki natural structure use karo — markdown headers, HTML tags, code blocks. Each section becomes a chunk.

```
from langchain.text_splitter import MarkdownHeaderTextSplitter

headers = [('H1', 'h1'), ('H2', 'h2'), ('H3', 'h3')]
md_splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers)
chunks = md_splitter.split_text(markdown_text)
# Each chunk carries header info in metadata!
```

(d) Semantic chunking

Embedding-based. Adjacent sentences ka similarity calculate karo; jaha semantic shift ho, waha break karo. (Detailed coverage in Part 2.)

(e) Token-based

Character counts misleading hain — different scripts (Hindi, Chinese) me char-to-token ratio different hota hai. Production me **tiktoken** ya model-specific tokenizer use karo:

```
from langchain.text_splitter import TokenTextSplitter

splitter = TokenTextSplitter(
    chunk_size=256,          # tokens, not chars
    chunk_overlap=32,
    encoding_name='cl100k_base' # OpenAI's encoding
)
```

4.3 Chunk Size: The Tradeoffs

Chunk size	Pros	Cons	Good for
Small (100-300 tok)	Precise retrieval, less noise	Exposes surrounding context	FAQ, factoid QA
Medium (300-800 tok)	Balanced — default choice	Generic compromise	Most RAG apps
Large (800-2000 tok)	Rich context, fewer chunks	Diluted relevance signal	Narrative, reasoning tasks

■ TIP

Empirically: **chunk_size = 500-800 tokens, overlap = 10-20% of size** ek solid starting point hai. Phir tune karo apne eval set pe.

4.4 Overlap: Why and How Much

Overlap se context bleed hota hai — agar koi key sentence chunk boundary pe tha, dono adjacent chunks me appear karega. Isse retrieval miss hone ke chances kam hote hain.

Rule of thumb: 10-20% of chunk size. Zyada overlap = redundant chunks = vector DB bloat aur duplicate retrievals.

4.5 Special: Code & Tables

- **Code:** Language-aware splitter use karo (Python AST-based, etc.). Functions/classes ko break mat karo. LangChain me `Language.PYTHON` specify kar sakte ho `RecursiveCharacterTextSplitter` me.
- **Tables:** Pure table ko ek chunk rakhna best hai (smaller tables). Bade tables ke liye row-wise chunking with header replication.

5. Embeddings: Text to Vectors

Embedding = ek dense vector representation jo text ke *semantic meaning* ko capture karta hai. Similar meaning wale texts → similar vectors → high similarity score.

5.1 Quick Intuition

```
embed('dog')    → [0.21, -0.55, 0.81, ..., 0.13]    (768-dim)
embed('puppy')   → [0.19, -0.52, 0.78, ..., 0.15]    ≈ very similar
embed('car')      → [0.91, 0.33, -0.42, ..., 0.65]    ≈ different
```

Math behind it: model ek high-dimensional space banata hai jaha *semantic neighbors close hote hain*. Cosine similarity 2 vectors ke beech = meaning ki closeness.

5.2 Dense vs Sparse Embeddings

	Dense	Sparse
Example	OpenAI ada-002, BGE, E5	BM25, TF-IDF, SPLADE
Vector size	384-3072 (small, dense)	10K-100K+ (vocab-sized, mostly 0)
Captures	Semantic similarity	Exact term matching
Strength	'puppy' ≈ 'dog'	Acronyms, rare terms, IDs
Weakness	Out-of-domain words	Synonyms, paraphrases

■ KEY POINT

Production-grade RAG almost always **hybrid** hota hai — dense + sparse combined. Best of both. (Detail in Part 2.)

5.3 Popular Embedding Models (2026 landscape)

Model	Dim	Notes
OpenAI text-embedding-3-large	3072 (or smaller via MRE)	API, strong general
OpenAI text-embedding-3-small	1536	Cheaper, still great
BAAI bge-large-en-v1.5	1024	Open-source, top of MTEB
intfloat/e5-large-v2	1024	Open, well-balanced
jina-embeddings-v3	1024	Multilingual, long context (8K)
Cohere embed-multilingual-v3	1024	API, 100+ langs
voyage-large-2	1536	Strong on retrieval benchmarks
Nomic embed-text-v1.5	768	Open, fast, MRL-enabled

■ NOTE

MTEB Leaderboard (Massive Text Embedding Benchmark) check karo Hugging Face pe — ye current state-of-the-art ka best public ranking hai retrieval/clustering tasks pe.

5.4 Choosing an Embedding Model

- **Domain:** General-purpose (BGE, OpenAI) vs domain-specific (BioBERT for medical, FinBERT for finance).
- **Language:** English-only vs multilingual. Indian use case? Try multilingual-e5, BGE-M3, jina-embeddings-v3.
- **Latency budget:** API (OpenAI/Cohere) vs self-hosted. Self-hosted = no per-call cost but infra needed.
- **Dimensions:** Bigger $\neq$ always better. 768-1024 dim usually sufficient. Higher dim = more storage + slower search.
- **Max context:** Some models 512 tokens, others 8K+. Match with your chunk size.
- **MRL (Matryoshka):** Newer models let you truncate vectors (e.g., 1024 $\rightarrow$ 256) with graceful degradation. Great for storage vs accuracy tradeoff.

5.5 Code Example

```
# Open-source: sentence-transformers
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('BAAI/bge-large-en-v1.5')
vecs = model.encode(['What is RAG?', 'Define retrieval'])
# vecs.shape == (2, 1024)

# OpenAI
from openai import OpenAI
client = OpenAI()
resp = client.embeddings.create(
    model='text-embedding-3-small',
    input=['What is RAG?']
)
vec = resp.data[0].embedding # list of 1536 floats
```

5.6 Normalize Your Vectors!

Cosine similarity use kar rahe ho? Vectors ko L2-normalize karo. Then cosine = dot product = much faster. Most modern embedding APIs already normalized vectors return karte hain — documentation check karo.

```
import numpy as np
def l2_normalize(v):
    norm = np.linalg.norm(v, axis=-1, keepdims=True)
    return v / (norm + 1e-12)
```

6. Vector Databases & ANN Search

Tumhare paas millions of vectors hain — har query ke liye sab se cosine similarity calculate karna feasible nahi ($O(N)$ per query). Vector databases **Approximate Nearest Neighbor (ANN)** indexes use karte hain jo $O(\log N)$ ya better deliver karte hain, thodi accuracy ka cost lekar.

6.1 Major Indexing Algorithms

HNSW (Hierarchical Navigable Small World)

Graph-based. Multi-layer structure — upar sparse, neeche dense. Search top layer pe shuru hoti hai (long jumps), neeche refine hoti hai. **Industry default** currently.

- High recall, low latency, supports filtering
- High memory usage (graph stored in RAM typically)
- Params: M (connections per node, ~16-64), ef (search candidates, higher = more accurate but slower)

IVF (Inverted File Index)

Vectors ko clusters (cells) me partition karta hai (k-means). Query ke time sirf nearby cells search hote hain.

- Memory efficient, scales to billions
- Lower recall than HNSW typically; requires training
- Params: nlist (number of cells), nprobe (cells to search per query)

PQ (Product Quantization)

Compression technique — vectors ko sub-vectors me todke quantize karta hai. Memory dramatically reduce hota hai (8-32x). Often combined with IVF → **IVF-PQ**.

LSH (Locality-Sensitive Hashing)

Hash functions design karte hain jaha similar vectors same bucket me jaate hain. Older approach — modern systems me less common but conceptually important.

ScaNN (Google)

Anisotropic quantization use karta hai. Very fast on Google's infra. Used in Vertex AI Matching Engine.

6.2 Distance Metrics

Metric	Formula intuition	When to use
Cosine	Angle between vectors (magnitude-invariant)	Default for normalized embeddings
Dot product	Sum of element-wise products	Same as cosine if vectors are normalized
Euclidean (L2)	Straight-line distance	Image embeddings, some specific models
Manhattan (L1)	Sum of absolute differences	Rare — sparse / high-dim cases

■ NOTE

Embedding model ke documentation me check karo recommended metric kya hai. Mismatch matlab garbage results.

6.3 Popular Vector Databases

DB	Type	Strengths	Best for
FAISS	Library	Fastest, Facebook-built	Research, in-process
Chroma	Embedded/server	Dev-friendly, simple API	Prototyping, small apps
Qdrant	Server (Rust)	Filtering, payload, scaling	Production, open-source
Weaviate	Server	Modules, hybrid search built-in	Multi-modal, GraphQL fans
Pinecone	Managed cloud	Zero-ops, fast	Enterprise, no infra team
Milvus	Server (distributed)	Massive scale	Billions of vectors
pgvector	Postgres ext.	Works with existing PG	Already on Postgres
Elasticsearch	Hybrid search	Mature filtering, BM25	Existing ES users
LanceDB	Embedded (Rust)	On-disk, fast, serverless-friendly	Edge / serverless

6.4 Code Example: Chroma + LangChain

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')

# Index docs
vectordb = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='./chroma_db'
)

# Query
results = vectordb.similarity_search('What is HNSW?', k=4)
for r in results:
    print(r.page_content[:200])
    print(r.metadata)
```

6.5 Code Example: Qdrant (production-friendly)

```
from qdrant_client import QdrantClient
from qdrant_client.models import VectorParams, Distance, PointStruct

client = QdrantClient(url='http://localhost:6333')

client.create_collection(
    collection_name='docs',
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
)

client.upsert(
    collection_name='docs',
    points=[
        PointStruct(id=i, vector=vec, payload={'text': txt, 'src': src})
        for i, (vec, txt, src) in enumerate(data)
    ]
)

hits = client.search(
    collection_name='docs',
    query_vector=query_vec,
    limit=5,
    query_filter={'must': [{'key': 'src', 'match': {'value': 'manual.pdf'}}]}
)
```

7. Retrieval Strategies

7.1 Vanilla Similarity Search

Simplest. Query ko embed karo, top-k nearest neighbors le aao. k usually 3-10.

```
results = vectordb.similarity_search(query, k=5)
# Returns 5 most similar chunks
```

7.2 Maximal Marginal Relevance (MMR)

Naive top-k ka problem: top results often **redundant** hote hain — same info different chunks me. MMR diversity introduce karta hai by balancing:

$$\text{MMR}(d) = \lambda \cdot \text{sim}(d, \text{query}) - (1 - \lambda) \cdot \max_{d' \in \text{selected}} \text{sim}(d, d')$$

$\lambda = 1.0$ → pure relevance (= top-k)
 $\lambda = 0.0$ → pure diversity
 $\lambda = 0.5$ → balanced (common default)

```
results = vectordb.max_marginal_relevance_search(
    query, k=5, fetch_k=20, lambda_mult=0.5
)
```

■ TIP

MMR specially useful jab tumhara corpus me near-duplicates hain (FAQ pages, scraped data). Wastage avoid hota hai context window me.

7.3 Metadata Filtering

Vector similarity ke saath structured filters apply karo. Critical in production.

```
# Only search in 2024 docs from finance dept
results = vectordb.similarity_search(
    query,
    k=5,
    filter={'year': 2024, 'dept': 'finance'}
)
```

■ NOTE

Pre-filter vs post-filter: production DBs (Qdrant, Weaviate) **pre-filter** support karte hain — index search se pehle apply hota hai → fast aur accurate. Naive systems post-filter karte hain → kam results aur k consistent nahi rehta.

7.4 Threshold Filtering

Sometimes top-k results irrelevant hote hain agar query out-of-distribution hai. Similarity score threshold lagao:

```
# Only keep results with cosine sim > 0.7
results = vectordb.similarity_search_with_score(query, k=10)
results = [d for d, score in results if score > 0.7]
```

■■ WARNING

Threshold tuning empirical hai — embedding model aur domain ke saath vary karta hai. Eval set pe calibrate karo, hardcoded magic numbers se bachao.

7.5 Multi-Vector Retrieval (preview)

Ek document ke multiple representations store karo — full text, summary, hypothetical questions. Query against summaries, return full text. Advanced topic — Part 2 me detail.

8. Generation: Prompting the LLM

8.1 Standard RAG Prompt Template

You are a helpful assistant. Answer the user's question using ONLY the provided context. If the answer is not in the context, say "I don't have enough information to answer this."

Context:

{retrieved_chunks}

Question: {user_query}

Answer (cite sources using [chunk_id]):

8.2 Prompt Engineering Tips

- **Explicit grounding instruction:** 'Use only the context.' Without this, models freely hallucinate.
- **Refusal instruction:** 'If not in context, say you don't know.' Otherwise model bana ke jawab dega.
- **Citation requirement:** Ask for source attribution → improves verifiability and reduces hallucination.
- **Chunk delimiters:** Use clear separators (---, <doc> tags). Helps model parse boundaries.
- **Numbered chunks:** [1], [2], [3]... → easier for citations and debug.
- **Order matters:** Most LLMs show *lost-in-the-middle* — info at start/end weighted more. Put most relevant chunk at the end (closer to query).

8.3 Code Example: End-to-end with LangChain

```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

retriever = vectordb.as_retriever(search_kwargs={'k': 4})
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

template = '''Answer based on context only.
Context: {context}
Question: {question}
Answer:'''
prompt = ChatPromptTemplate.from_template(template)

def format_docs(docs):
    return '\n\n'.join(
        f'[{i+1}] {d.page_content}' for i, d in enumerate(docs))

chain = (
    {'context': retriever | format_docs,
     'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

answer = chain.invoke('What is HNSW?')
```

8.4 Temperature & Other Params

- **Temperature = 0** for RAG. Tum factual answer chahte ho, creativity nahi.
- **Max tokens:** Set generously (1-2K). Truncated answers bure hote hain.
- **Streaming:** User experience ke liye essential — pehle hi tokens dikhne lagte hain.

9. Evaluation: Are We Even Good?

Tum RAG bana liya — par kaise jaano ye actually accha hai? RAG evaluation do parts me decompose hoti hai: **retrieval quality** aur **generation quality**.

9.1 Retrieval Metrics

Recall@k

Top-k retrieved chunks me se kitne actually relevant the (vs ground truth)? $\text{recall@k} = |\text{retrieved} \cap \text{relevant}| / |\text{relevant}|$

Precision@k

Top-k me se kitne percent relevant? $\text{precision@k} = |\text{retrieved} \cap \text{relevant}| / k$

MRR (Mean Reciprocal Rank)

First relevant document kis position pe aaya? $\text{MRR} = 1/\text{rank_first_relevant}$ — averaged over queries. 1.0 = always first, 0.5 = average position 2, etc.

NDCG (Normalized Discounted Cumulative Gain)

Position-aware ranking quality. Top positions ko zyada weight deta hai. Standard IR metric — order matter karta hai jab tum graded relevance rakhte ho (highly relevant > somewhat relevant > irrelevant).

Hit Rate

Simplest: kya at least 1 relevant chunk top-k me aaya? Binary 0/1, averaged over queries.

9.2 Generation Metrics

- **Faithfulness / Groundedness:** Generated answer context me supported hai? Hallucinated to nahi?
- **Answer relevance:** Answer query ke direct point pe hai?
- **Answer correctness:** Vs ground truth — semantic similarity / exact match / LLM judge.
- **Context precision:** Retrieved chunks me se kitne actually use hue?
- **Context recall:** Ground truth answer ke saare facts retrieved chunks me hain?

9.3 RAGAS Framework (Quick Intro)

RAGAS = library jo ye sab metrics calculate karti hai, LLM-as-judge se. Detailed coverage Part 2 me, but basic usage:

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall
)

dataset = Dataset.from_dict({
```

```
'question': questions,
'answer': generated_answers,
'contexts': retrieved_contexts,
'ground_truth': true_answers
})

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy,
             context_precision, context_recall]
)
```

9.4 Building Your Eval Set

- **Synthetic generation:** LLM se questions generate karwao from your corpus. Fast but quality variable.
- **Human-curated:** SMEs likhte hain Q-A-source triples. Gold standard, slow.
- **Production logs:** Real user queries + thumbs-up/down. Most representative.
- Aim for **50-200 high-quality examples** minimum before measuring anything serious.

■ KEY POINT

'You can't improve what you can't measure.' — RAG pipeline tune karne se pehle eval set bana lo. Warna har change after-the-fact rationalization hai.

10. Putting It All Together

Ek complete minimal RAG pipeline — ingestion se response tak — under 50 lines:

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# 1. Load
docs = PyPDFLoader('handbook.pdf').load()

# 2. Split
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800, chunk_overlap=120
)
chunks = splitter.split_documents(docs)

# 3. Embed + Index
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
db = Chroma.from_documents(chunks, embeddings,
                           persist_directory='./db')

# 4. Retriever
retriever = db.as_retriever(
    search_type='mmr',
    search_kwargs={'k': 4, 'fetch_k': 15, 'lambda_mult': 0.5}
)

# 5. Prompt + LLM
prompt = ChatPromptTemplate.from_template('''
Answer using ONLY the context. Cite chunk numbers like [1].
If unsure, say 'I don\'t know'.

Context:
{context}

Question: {question}
Answer:''')

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

def fmt(docs):
    return '\n\n'.join(f'[{i+1}] {d.page_content}'
                       for i, d in enumerate(docs))

# 6. Chain
chain = (
    {'context': retriever | fmt,
     'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

# 7. Query
print(chain.invoke('What is the leave policy?'))
```

11. Common Pitfalls (Aaj Hi Sikho)

- **Chunk size = 1000 hardcoded:** Optimize per use case.
- **Same embedding for indexing aur query nahi:** Always match.
- **No metadata stored:** Filtering, debugging — sab kharab.
- **No eval set:** 'Trust me bro' development.
- **Top-k = 1:** Almost always too few. Start with 4-5.
- **Naive PDF parsing:** Tables, headers ko ignore karke garbage chunks bana dete ho.
- **Temperature > 0 in production RAG:** Hallucinations multiply.
- **Forgetting to handle 'no good results':** Retrieved chunks irrelevant ho to graceful refusal hona chahiye, na ki LLM ko hallucinate karne ka mauka.

12. What's Next?

Tumne base RAG cover kar liya — ingest, chunk, embed, retrieve, generate, evaluate. Ye 80% real-world projects ka starting point hai.

Part 2 (Advanced RAG) me jaayenge: query transformations (HyDE, multi-query, step-back), hybrid search, re-ranking, Self-RAG, Corrective RAG, GraphRAG, multimodal, aur production-grade tuning.

Part 3 (Agentic RAG) me: agents, ReAct, routing, adaptive RAG, LangGraph patterns, iterative retrieval.

■ KEY POINT

Stanford CS336 ke transformer foundations + ye RAG series + ek hands-on project (e.g., chat-over-your-notes app) — interview portfolio ke liye solid combo. Build it, ship it on HuggingFace, write a blog. Recruiters yahi dekhte hain. ■